



EJERCITACIÓN — CICLOS

Objetivo y propósito de la sesión

Objetivo: aplicar `for+range()`, `for` anidado, `while` y `continue/break` en problemas estilo evaluación, para llegar con seguridad a la Evaluación de Ciclos del jueves.

Propósito: hoy practicamos bajo las condiciones de la evaluación del jueves — mismos tipos de problemas, mismo trabajo individual, pero sin nota. Practicar bajo esa presión controlada, y revisar juntos los errores al final, es lo que te permite llegar con más confianza al día real.

Nombre: _____
 Fecha: _____
 Curso: _____

Instrucciones generales

- Esta sesión tiene **3 partes**: una Práctica Guiada (se resuelve en conjunto), 4 ítems cortos, y 2 programas de desarrollo.
- **No lleva nota.** Al terminar, revisamos juntos los errores más comunes.
- Escribe tu código **a mano, con letra clara** — el profesor debe poder leerlo sin dudas.
- Si te equivocas, tacha con una línea y escribe al lado. No necesitas que quede perfecto, pero sí legible.
- Usa nombres de variables en **snake_case** en español.
- Cuando un ítem pida un dato con `input()`, el enunciado siempre aclara si es un **número entero**, un **número con decimales**, o una **palabra exacta**.
- Si no terminas un ítem, deja lo que alcanzaste — se revisa lo escrito tal como está.
- Trabajo individual, en las mismas condiciones que la evaluación real.

Regla de indentación

Las líneas punteadas marcan cada nivel de anidación. Alinea tu código con la raya de su nivel, como en el ejemplo:

```
for i in range(5):
    print(i)
    if i == 3:
        print("tres")
```

Práctica Guiada — se resuelve en conjunto

Torneo de videojuegos — el puntaje más alto. Un torneo de videojuegos registra los puntajes de la ronda de clasificación. Cada jugador ingresa su puntaje, uno por uno, y el sistema debe seguir recibiendo puntajes hasta que se ingresa **-1**, que marca el fin de la ronda. Al terminar, el sistema debe informar cuál fue el **puntaje más alto** registrado en toda la ronda.

El programa debe:

- Pedir puntajes repetidamente con **while**, hasta recibir **-1**.
- Ir comparando cada puntaje ingresado con el más alto registrado hasta el momento, y quedarse con el mayor.
- Imprimir el puntaje más alto al terminar.
- Si el primer ingreso ya es **-1** (no se registró ningún puntaje), informar que no hubo puntajes, sin mostrar un máximo.

Pista — Cómo partir el máximo: Antes de tener ningún puntaje, todavía no existe un “máximo”. Puedes usar el primer puntaje válido que se ingrese como punto de partida, y desde ahí ir comparando los que vengán después.

Ejemplo 1	Ejemplo 2
Entrada: 850, 1200, 690, -1	Entrada: -1
Salida: Puntaje más alto: 1200	Salida: No se registró ningún puntaje.

Escribe el programa acordado en clase aquí:

Sección 1 — Ítems cortos

Repaso rápido de ciclos, dividido en dos partes.

1A — Arma el código

Completa la parte marcada con líneas en blanco (_____) para que el programa haga lo que pide la narrativa. Escribe directamente sobre la línea, en el mismo bloque de código.

Ítem 1A.1. La radio escolar transmite un anuncio publicitario cada 3 minutos, desde el minuto 0 hasta el minuto 21 de un bloque de transmisión. El programa debe imprimir en qué minutos se transmite un anuncio.

```
for minuto in _____:  
    print("Anuncio en el minuto", minuto)
```

***Pista:** Un `range()` de tres números es `range(inicio, fin, paso)` — recuerda que el fin queda excluido, así que debe ser un número más que el último minuto que quieres incluir.*

Ítem 1A.2. El sistema de un parque de diversiones revisa las atracciones numeradas del 1 al 12, buscando la primera que tiene mantenimiento programado: la atracción N°9. Apenas la encuentra, debe avisar cuál es y **detener la búsqueda de inmediato**, sin seguir revisando las que quedan.

```
for atraccion in range(1, 13):  
    if atraccion == 9:  
        print("Atracción en mantenimiento: N°", atraccion)  
        _____
```

***Pista:** Piensa en la instrucción que corta un ciclo de inmediato, sin esperar a que termine de recorrer los valores que quedan.*

1B — Arregla el bug

Cada fragmento tiene un solo error. Reescribe el código **corregido y completo** en el espacio en blanco debajo de cada uno.

Ítem 1B.1. Un campamento de verano organiza 3 cabañas con 4 actividades cada una. El sistema debe imprimir cada actividad de cada cabaña, pero algo salió mal: solo se imprime la última actividad de cada cabaña.

```
for cabana in range(1, 4):
    for actividad in range(1, 5):
        numero_actividad = actividad
        print("Cabaña", cabana, "- Actividad", numero_actividad)
```

Pista: Cuenta cuántas veces se ejecuta el `print()` tal como está escrito, y compáralo con cuántas veces debería ejecutarse según el enunciado.

Código corregido:

Ítem 1B.2. Una app de estudio revisa 8 flashcards numeradas del 1 al 8 y debe contar cuántas son de **repaso rápido**: las de número **par**. Las impares se saltan con **continue**. Pero el contador de repaso rápido nunca sube como corresponde — siempre termina en 8.

```
contador_repaso_rapido = 0

for numero_flashcard in range(1, 9):
    contador_repaso_rapido = contador_repaso_rapido + 1
    if numero_flashcard % 2 != 0:
        continue

print("Repaso rápido:", contador_repaso_rapido)
```

Pista: *Revisa el orden: ¿la línea que suma el contador se ejecuta antes o después de que el programa decida si esa flashcard se salta?*

Código corregido:

Sección 2 — Programas completos

Programas completos desde una narrativa. El Ejercicio 2 pide sus datos con `input()` — lee bien el tipo de dato que se espera antes de escribir tu código.

Ejercicio 1 — Feria de talentos. La feria de talentos del liceo tiene 4 bloques de presentaciones. El bloque i tiene i presentaciones, y cada presentación dura 8 minutos. El programa debe calcular el **total de minutos** de presentaciones en toda la feria, y además contar en **cuántos bloques se superaron los 24 minutos** de duración.

El programa debe:

- Usar `bloques_totales = 4`.
- Para cada bloque (empezando en 1), calcular los minutos de ese bloque sumando 8 minutos por cada presentación que le corresponde, y acumular ese subtotal en el total general de la feria.
- Contar en cuántos bloques la duración del bloque superó los 24 minutos (estrictamente mayor).
- Imprimir el total de minutos de toda la feria y la cantidad de bloques que superaron los 24 minutos, cada uno con su etiqueta.

Pista 1 — Reinicia el contador de minutos en cada bloque: Antes de empezar a contar las presentaciones de un bloque nuevo, la variable que acumula los minutos de ese bloque debe volver a partir en 0.

Pista 2 — El contador de bloques se revisa después del ciclo interno: Recién cuando terminaste de sumar todas las presentaciones de un bloque sabes si ese bloque superó los 24 minutos.

El programa imprime:

```
Total de minutos de la feria: 80
Bloques que superaron los 24 minutos: 1
```

Escribe tu programa aquí:

Ejercicio 2 — Invernadero escolar. Un invernadero escolar registra la temperatura de sus primeros 6 sensores del día, ingresada manualmente uno por uno. El programa debe contar cuántas lecturas fueron “**normales**” (entre 18 y 28 grados, ambos límites incluidos) y cuántas fueron “**de alerta**” (fuera de ese rango).

El programa debe:

- Usar un **for** con **range(6)** para repetir exactamente 6 veces.
- Por cada vuelta, pedir la temperatura del sensor correspondiente con **input()** (puede tener decimales).
- Clasificar cada lectura como normal o de alerta, según el rango indicado.
- Imprimir ambos conteos al terminar, cada uno con su etiqueta.

***Pista 1 — Dos contadores, uno por categoría:** Necesitas una variable que cuente las lecturas normales y otra distinta que cuente las de alerta — cada lectura suma a una sola de las dos.*

***Pista 2 — Ambos límites incluidos:** Una lectura de exactamente 18 o exactamente 28 grados cuenta como normal, no como de alerta.*

Ejemplo 1	Ejemplo 2
Entrada: 20, 30, 18, 28, 15, 22	Entrada: 15, 35, 20, 25, 10, 40
Salida: Lecturas normales: 4 Lecturas de alerta: 2	Salida: Lecturas normales: 2 Lecturas de alerta: 4

Escribe tu programa aquí: